



Portfolio: Part III : Language Task

Deep Learning-based Fake News Classification using RNN, LSTM, and Pretrained GloVe Embeddings

Name: Rishant Jung Karki

Student ID: 2408591

Module: Artificial Intelligence & Machine learning

Module Leader: Mr. Siman Giri

Tutor: Ms. Sunita Parajuli

Contents

1. Abstract.....	1
2. Introduction	1
3. Dataset	2
3.1 Source	2
3.2 Data Size and Inspection.....	2
3.3 Preprocessing	3
4. Methodology	3
4.1 Tokenization, Encoding, and Padding.....	3
4.2 Model Architectures	4
Model 1 — Simple RNN with Trainable Embedding	4
Model 2 — LSTM with Trainable Embedding	4
Model 3 — LSTM with Pretrained GloVe Embeddings	5
4.3 Loss Function and Optimiser.....	5
4.4 Hyperparameters.....	7
5. Experiments and Results	8
5.1 Training Performance	8
5.2 Test Set Evaluation	9
5.3 Confusion Matrix and Classification Report	9
5.4 Computational Efficiency	10
5.5 Embedding Comparison	11
6. Error Analysis.....	11
7. Real-Time Prediction Interface	12
8. Conclusion and Future Work.....	12
GitHub Link for Code:.....	13

1. Abstract

This study proposes an approach which uses RNN, LSTM, and pre-trained GloVe embeddings to detect fake news. The main aim is to develop a classifier that classifies news into fake news and real news using natural language processing (NLP). There has been a growing need for detecting fake news since fake news is spreading rapidly across various digital channels and social media, which is causing confusion among the people.

The project adopts a fully-fledged NLP pipeline starting from data preprocessing, cleaning, tokenization, sequence padding, model design, training, evaluation, and live prediction. In total, three deep learning models were designed and tested: Simple RNN with trainable embeddings, LSTM with trainable embeddings, and LSTM with pre-trained GloVe embeddings.

Data was cleaned using strategies like converting to lowercase, replacing contractions, removing URL links, removing punctuation marks, removing stop words, and lemmatizing the words. Finally, the text data was tokenized and padded to a fixed length of 300 tokens using Keras Tokenizer.

From the experimental results obtained, it can be observed that the best test accuracy among all was provided by LSTM model using pretrained GloVe embeddings, gaining an impressive score of 98.55%. Simple RNN and LSTM performed relatively poorly, providing accuracies of 94.42% and 92.70%, respectively.

2. Introduction

Natural Language Processing (NLP) is considered as one of the important domains of Artificial Intelligence. It's aim is to develop computer programs capable of understanding and interpreting natural languages. A common use case of NLP is text classification, which includes categorizing textual content into predetermined classes.

In recent times, the detection of fake news has been a serious problem, particularly on digital platforms, as it can be quickly shared through the internet. Fake news can have a harmful effect on people's opinions, political systems, healthcare awareness, and social peace. For this reason, developing automated fake news classification systems has become essential.

Typically, text classification applications employ traditional machine learning algorithms. Such algorithms involve developing features manually, using Bag-of-Words or Term Frequency Inverse Document Frequency. While these techniques may deliver acceptable accuracy, they fail to account for sequential and context-sensitive connections between the words.

Conversely, neural network-based deep learning frameworks such as Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) networks perform better on sequential data. The advantage of LSTM networks is that they automatically learn contextual information from the texts. On the other hand, conventional RNN models experience vanishing gradient issues while processing long sequences.

This project involves developing several different architectures based on deep learning for the detection of fake news. The tasks include:

- Data pre-processing for textual data.
- Implementation of deep learning models based on RNN and LSTM.
- Comparison of trainable and GloVe embeddings.
- Evaluation of models using multiple metrics.
- Development of a Gradio application for prediction.

3. Dataset

3.1 Source

The dataset used in this study includes "True vs Fake News Data Set" loaded from a CSV file using the Pandas library. The data set consists of news articles which are classified into either fake news or true news. And two major attributes including "News Article" and "Label." The "Label" attribute was converted numerically with labels of "Fake" = 0 and "True" = 1.

3.2 Data Size and Inspection

The dataset contains several thousand news articles that are classified into two groups: Fake News and True News. The notebook conducted dataset inspection involving the total number of samples, missing values inspection, distribution of classes, and unique labels inspection. Fake and true news distribution was visualized using pie charts and bar charts.

3.3 Preprocessing

The text preprocessing step was carried out to clean and standardize the raw textual data before training the models. Text preprocessing involves the following stages in sequence:

- Conversion of all text to lowercase
- Expansion of contractions (e.g. "don't" → "do not") using the contractions library
- Removing URLs (http/www patterns)
- Removing email addresses
- Removing @mentions
- Removing #hashtags
- Removing numbers
- Removing punctuation and special characters
- Removing extra whitespace
- Tokenization, stopword removal, and lemmatization using NLTK WordNetLemmatizer

The cleaned text was stored in a new column with the name “cleaned_text”. Visualizations of word clouds and analysis of the top 20 most frequently used words were done separately for fake news and true news articles.

4. Methodology

4.1 Tokenization, Encoding, and Padding

The clean text was tokenized by the Keras Tokenizer with the following parameters: maximum vocab size = 10,000 tokens and out-of-vocab token <OOV>. The tokenizer was fitted on training data only, avoiding data leakage.

The dataset was divided into 80/20 i.e. 80% training data and 20% testing data using a stratified split (random_state=42) to ensure balanced class representation in both sets.

The length distribution for each sequence was analyzed. As a result of this analysis, MAX_LEN = 300 has been chosen as the maximum sequence length. Post padding and post truncation to length MAX_LEN = 300 were applied to all input sequences.

4.2 Model Architectures

Model 1 — Simple RNN with Trainable Embedding

The first implementation was done using a Simple Recurrent Neural Network that included an embedding layer. This is the architecture of this neural network:

- Input layer (sequence length 300)
- Embedding layer (vocab_size $\times$ 64 dimensions, trainable)
- SimpleRNN layer (64 units)
- Dropout layer (rate = 0.3)
- Dense layer (32 units, ReLU activation)
- Dropout layer (rate = 0.3)
- Dense output layer (1 unit, Sigmoid activation)

Word embeddings were obtained by the embedding layer. Information of tokens in sequence was captured using the SimpleRNN layer. Dropout layers were applied after the RNN and Dense layers to avoid overfitting. The output from the final sigmoid neuron is the probability of the binary classification task (0 for fake and 1 for true).

Advantage: Training process is more efficient with lower computational complexity.

Limitation: Difficult to preserve long-term context due to vanishing gradients issue.

Model 2 — LSTM with Trainable Embedding

The second model applied a Long Short-Term Memory network with a trainable embedding layer. The architecture is similar to that of Model 1, except the SimpleRNN layer which was substituted with an LSTM layer:

- Input layer (sequence length 300)
- Embedding layer (vocab_size $\times$ 64 dimensions, trainable)
- LSTM layer (64 units)
- Dropout layer (rate = 0.3)
- Dense layer (32 units, ReLU activation)

- Dropout layer (rate = 0.3)
- Dense output layer (1 unit, Sigmoid activation)

The LSTM network is built to tackle the vanishing gradient issue through memory cells and three gates: input, forget, and output. This enables the system to keep or throw away information during the process, which is necessary for reading news articles.

Model 3 — LSTM with Pretrained GloVe Embeddings

In the third model, pretrained GloVe word embeddings were integrated into an LSTM network architecture. Gensim was used to load the GloVe word embeddings using the glove-wiki-gigaword-50 model, which contains word vectors with 50 dimensions and have been trained using the Wikipedia and Gigaword corpora.

A matrix of size (vocab_size x 50) was created mapping each word in the tokenizer vocabulary to its corresponding GloVe vector. Words not found in GloVe were left as zero vectors.

Following is the architecture:

- Input layer (sequence length 300)
- Embedding layer (vocab_size × 50, weights initialized from GloVe matrix, trainable = False, mask_zero = True)
- LSTM layer (64 units)
- Dropout layer (rate = 0.3)
- Dense layer (32 units, ReLU activation)
- Dropout layer (rate = 0.3)
- Dense output layer (1 unit, Sigmoid activation)

The embedding weights were frozen (trainable=False) so the pretrained GloVe vectors were preserved during the training of the model. The mask_zero=True parameter instructed the model to ignore padding tokens, improving sequence representation quality.

4.3 Loss Function and Optimiser

The Binary Cross Entropy loss function was used in all three models since the problem involves binary classification where the output is either fake (0) or true (1). The Binary Cross Entropy calculated the error between the predicted probability values and their corresponding true labels.

Adam Optimizer was also used in all three models. The Adam optimizer is an extension of Momentum and RMSProp.

4.4 Hyperparameters

Hyperparameter	Value
Epochs	20 (with early stopping)
Batch Size	64
Embedding Dimension (trainable)	64
GloVe Embedding Dimension	50
Maximum Vocabulary Size	10,000
Maximum Sequence Length (MAX_LEN)	300
Validation Split	10% of training data
Early Stopping Monitor	val_loss
Early Stopping Patience	3 epochs
Early Stopping restore_best_weights	True
Dropout Rate	0.3 (applied twice per model)
Train / Test Split	80% / 20%

Early stopping monitored validation loss with a patience of 3 epochs and automatically restored the best model weights, preventing overfitting while reducing unnecessary training time.

5. Experiments and Results

5.1 Training Performance

All three models were trained on the same padded training dataset with the use of Adam Optimizer and Binary Cross-Entropy Loss.

For the accuracy and losses of all three models, training and validation curves were created with the help of the `plot_history` function.

The Simple RNN model demonstrated good training results but was a bit unstable in its validation losses because of the problem with capturing long-term dependency in a 300-token sequence.

As expected, the LSTM model performed much better in terms of validation losses and achieved better accuracy, which confirmed the advantage of gated cells in sequential data classification.

Finally, the best results in terms of convergence speed and stability were observed in the LSTM with GloVe embedding model, because of the initialization of the semantic space.

5.2 Test Set Evaluation

On the held-out 20% test set, all models were evaluated. Predictions were made by using a threshold of 0.5 on the sigmoid output probability. The following test accuracies were achieved:

Model	Architecture	Embedding	Test Accuracy
Model 1	Simple RNN	Trainable (64-dim)	94.42%
Model 2	LSTM	Trainable (64-dim)	92.70%
Model 3	LSTM	GloVe Frozen (50-dim)	98.55%

Model 3 (LSTM with GloVe) showed the highest test accuracy with 98.55% compared to Model 1 which gave 94.42% and Model 2 which gave 92.70%. Interestingly, Model 2 (LSTM with trainable embeddings) showed less accurate results than Model 1 (Simple RNN). Perhaps, it is related to the need for additional data in order for LSTM with trainable embeddings to show better results than simple models.

5.3 Confusion Matrix and Classification Report

Confusion Matrices and Classification Reports were gained for each model using the `evaluate_model` function.

The code includes 'Ham' and 'Spam' as labels in the classification report and confusion matrix axis labels. However, since the data consists of Fake and True news, the use of these labels creates an inconsistency between the naming and classification process, even though the output of the code is correct. The confusion matrix can be understood as: predicted and actual Fake (Class 0) when labelled as 'Ham' and Predicted and actual True (Class 1) when labelled as 'Spam'.

After analysis of the results of the confusion matrix for all three models, it can be seen that Model 3 has the least number of misclassifications compared to Models 1 and 2.

5.4 Computational Efficiency

The three different models have their differences in terms of computation:

- Simple RNN: The fastest training period with the lowest memory requirements and lowest computation complexity due to the simplicity of its recurrent architecture.
- LSTM: Slower than SimpleRNN because of four gating processes done per timesteps; more memory intensive but does a better job of context learning.
- LSTM+GloVe: Similar training time as LSTM for the forward/backward process; more preprocessing time is required to generate the GloVe matrix but it converges faster because the pre-trained embeddings are already a good place to start from.

All the models were processed with Google Colab GPU to decrease computation time.

5.5 Embedding Comparison

There were two types of embeddings used in this experiment: randomly initialized trainable embeddings and fixed pretrained GloVe embeddings.

Trainable embeddings created their own representation of words based on the training set. Despite being effective, they took more time to train and provided poor representations of rare/unseen words.

The advantage of the use of pretrained GloVe embeddings was that they already had semantic structure after being trained on an external dataset. Among the advantages there is improved word semantics, fast and stable convergence, higher test accuracies, and generalization to unseen words due to dense vectors.

6. Error Analysis

The errors have been analyzed with respect to the misclassified samples from Model 3 (LSTM + GloVe). The test set included the original texts, true labels, as well as predictions of all three models.

The types of errors can be classified as follows:

- False Positives — articles which belong to the True class and are predicted as Fake by the model
- False Negatives — articles which belong to the Fake class but are predicted as True

Possible explanations of the error include:

- The presence of conflicting information in an article, making its classification ambiguous.
- The presence of too much or noisy information within an article which is trimmed down to only 300 tokens.
- The presence of words that are not present in GloVe vocabulary and get zero vector representation.
- Class imbalance issues within the dataset.
- Limitations of LSTM — inability to perform deep logical reasoning and make sense out of complex facts.

Improvements based on the results of error analysis include using class weights, Bidirectional LSTM, trainable GloVe embeddings, and adding an attention mechanism. Also, transformer models (such as BERT) could be used for improvement purposes.

7. Real-Time Prediction Interface

An interactive web page using the Gradio library was designed as a supplementary feature. The `predict_fake_news()` function cleans the user-submitted news text with the identical preprocessing pipeline, tokenises and pads the input, and subsequently evaluates it using all three models in parallel.

The output from each model consists of two parts: a label (either FAKE or TRUE) and the percentage of probability that the input text is fake news. The results from all three models are displayed together, enabling users to compare the outcomes. Some examples of news texts are provided within the Gradio interface.

8. Conclusion and Future Work

The proposed method has successfully applied deep learning to classify fake news using SimpleRNN, LSTM, and LSTM with pre-trained GloVe embeddings. The findings have confirmed that the LSTM network performs better than the Simple RNN architecture due to its capability to maintain the long-term context with memory units and gate functions.

In terms of overall performances, Model 3 has yielded the highest accuracy with 98.55%, followed by Simple RNN and LSTM with 94.42% and 92.70%, respectively. The use of pre-trained embedding vectors has played an important role in semantic interpretation, convergence rate, and generalization capabilities.

Limitations were found:

- Overfitting issues at the initial stage of training for Models 1 and 2
- Costly computation involved in LSTM models
- Challenges to categorise unclear or mixed-type news articles
- Dependence on the GloVe vocabulary set for Model 3
- Inconsistency in labelling system for confusion matrix (Ham/Spam instead of Fake/True)

Future improvements may include:

- The usage of transformer-based models like BERT and RoBERTa for enhanced semantic comprehension
- Hyperparameter tuning (number of LSTM layers, dropout percentage, learning schedule)

- Training on extensive and varied corpora to improve generalisation ability
- Implementing bidirectional LSTM to analyse past and future contexts
- Spatial attention mechanisms to concentrate on significant portions of each text
- Making the model publicly accessible via an online application
- Converting the model into a multilingual fake news detector

Overall, this project provided practical experience in deep learning, NLP pipelines, text preprocessing, sequence modelling, and pretrained word embeddings for binary text classification tasks.

GitHub Link for Code:

<https://github.com/Rishant-Jung-Karki/AI-ML-Assessment>